

MDS5112 / CSC6051 – Image Processing and Computer Vision – Assignment 1

Teaching Assistant: Chen Shi
Email: 224040349@link.cuhk.edu.cn

1 Programming Environment and Hardware Requirements

All tasks in this assignment must be completed using Python. Python is widely used in machine learning and deep learning, with rich library and framework support. You may choose either PyTorch or TensorFlow as your deep learning framework. While the example code in this document is primarily based on PyTorch, you may choose the framework according to your preference. **Note that if you opt for PyTorch, you must use version 2.0 or higher.**

Model training for this assignment requires GPU acceleration. All experiments are configured for a single GPU. You have three options for GPU resources: a personal computer equipped with an NVIDIA GPU, the university's GPU cluster (if our application is approved), or the free GPU resources provided by Google Colab.

We are currently applying for access to the university's GPU cluster. If approved, each student will be able to submit jobs requesting up to 4 CPU cores and 1 NVIDIA 2080 Ti GPU. **Please note that these computational resources are shared among students from this course as well as other courses, and jobs will be queued accordingly.**

Google Colab (<https://colab.research.google.com>) is a free cloud-based Jupyter notebook environment that provides GPU support. It offers several advantages, including no need for a local GPU, pre-installed deep learning libraries and frameworks, and the ability to write and run code directly in the browser. When using Google Colab, please be aware that GPU usage time is limited for each session, a Google account is required for login, and it's recommended to save your code and data to Google Drive for easy management and persistence. **Additionally, we provide the training data on Google Cloud, which you can directly download in Colab by running:** `!gdown 1UvILBVosUAeMr3Ugr2L13a24ZsDT54iQ.`

If you choose to use a personal computer, ensure that your machine is equipped with an NVIDIA GPU, CUDA and cuDNN are correctly installed, and you have installed a GPU version of PyTorch or TensorFlow compatible with your CUDA version.

2 Task 1: Image Affine Transformation (10 Points)

2.1 Background

This section aims to implement a simple image affine transformation, only considering scale, rotation, and translation. Given an image I , a scale factor s , a rotation angle θ , and a pair of translation distance t_x and t_y , the implemented function means to output the transformed image $\hat{I}$ according to s , θ , t_x and t_y .

2.2 Todo List:

Implement the image transformation function with the parameters scale, rotation, and translation. The scale ranges from 0.0 to 2.0, the rotation angle ranges from 0 to 360 (anticlockwise), and the translation distance from -100 to 100. Some examples are shown in Figure 1.

2.3 Hints

There exist multiple solutions to realize the proposed target. Here we would like to recommend one approach which computes the affine transformation matrix first. Specifically, it consists of the following steps:

- Transform the given angle θ to radian ($\pi * \theta / 180$).
- Sample three points (a triangle) in the given image, and compute corresponding point coordinates in the target.

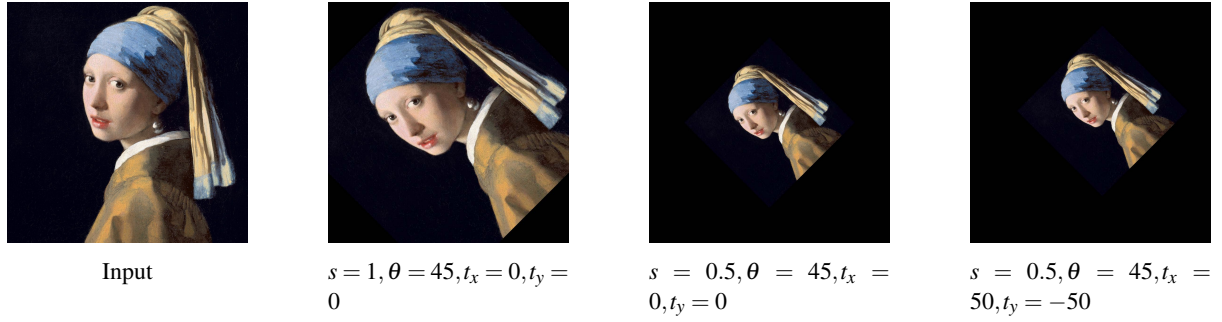


Figure 1: The visualization of our input and outputs.

- Calculate transformation matrix using six points with function `cv2.getAffineTransform()`.
- Obtain the target image by using transform matrix with function `cv2.warpAffine()`.

Important! If you follow the given approach to implement your program, you need to note the coordinate system used in the image rotation. When we process images by using `cv2` or other packages, the original point is located in the left-up corner of the image. Meanwhile, we usually conduct the rotation operation referring to the center of the image as the original point like Figure 2.

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)

cv2.imread

(-2,-2)	(-2,-1)	(-2,0)	(-2,1)	(-2,2)
(-1,-2)	(-1,-1)	(-1,0)	(-1,1)	(-1,2)
(0,-2)	(0,-1)	(0,0)	(0,1)	(0,2)
(1,-2)	(1,-1)	(1,0)	(1,1)	(1,2)
(2,-2)	(2,-1)	(2,0)	(2,1)	(2,2)

In Fact

Figure 2: The coordinate systems of the pixel location used in `cv2` (left) and image rotation. For better illustration, we assume the image size is 5×5.

3 Task2: Project 3D Points to Retina Plane (20 points)

3.1 Background

This section aims to use the pinhole camera model which performs a perspective projection. We will project a 3D cube onto a 2D camera plane using this model.

3.2 Todo List:

- Complete the function `get_camera_intrinsics` which returns a 3x3 camera matrix for provided focal lengths and the principal point. (5 points)
- Develop the function `get_perspective_projection`, which takes in a 3D point in camera space x_c and the camera matrix K and returns the point in screen space (i.e. the pixel coordinates) x_s . (5 points)
- Plot the projected cube, using the default settings as illustrated in Figure 3. (5 points)
- Experiment with different settings. (5 points)

- Cube orientation: Rotate the cube to observe changes in its projected shape
- Cube position: Adjust the cube's center coordinates relative to the camera
- Focal length: Modify the camera's focal length to simulate different fields of view

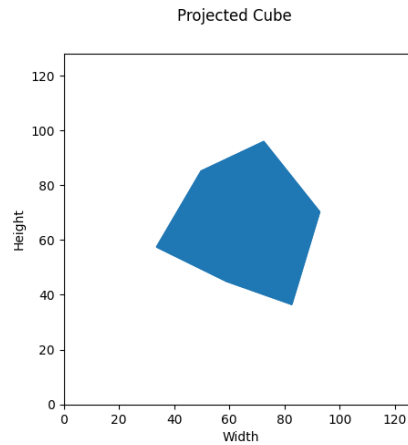


Figure 3: Perspective projection of a 3D cube. Parameters: cube center at $(0, 0, 2)$, rotation angles $[30^\circ, 50^\circ, 0^\circ]$, scale=1.0, camera focal lengths $\alpha = \beta = 70$ pixels and $c_x = c_y = 64$ pixels.

4 Task3: Portrait Segmentation (70 points)

4.1 Background

In this task, you are required to implement a portrait segmentation model. As illustrated in Figure 4, the input consists of RGB images, and the expected output is the corresponding segmentation masks that separate foreground portraits from the background. We have provided baseline code to help you get started. Please ensure that the PyTorch environment is properly installed before proceeding with the implementation.



(a) Input image



(b) Segmentation mask

Figure 4: Portrait segmentation example. (a) Input RGB image. (b) Expected segmentation output.

4.2 Todo List:

- **Environment Setup and Baseline Training. (15 points)**

Configure the environment correctly and run `train.py` to train the network successfully.

- **Training Analysis. (20 points)**

Complete 20 epochs of training and plot the following curves as a function of epoch number:

- Training loss
- Validation loss
- Validation pixel accuracy

Analyze and discuss the trend of each metric throughout the training process.

- **Performance Improvement. (20 points)**

Implement additional techniques to further improve segmentation performance. Possible approaches include, but are not limited to:

- Data augmentation strategies (e.g., rotation, flipping, color jittering)
- Using pre-trained networks from torchvision (e.g., `torchvision.models.segmentation.fcn_resnet50`)
- Learning rate scheduling (e.g., `ReduceLROnPlateau`, `CosineAnnealing`)
- Loss function modifications

Document your experiments and compare the results with the baseline model.

- **Face Detection and Image Preprocessing. (10 points)**

The baseline model assumes that portrait subjects are roughly centered in the image. However, real-world images often contain faces at various positions and scales, which may lead to degraded segmentation performance. You need to design and implement a face-aware preprocessing pipeline to improve robustness:

- Implement face detection to automatically locate faces in images (you may use any available Python library)
- Develop a preprocessing pipeline that crops and resizes images to center the detected face region before feeding them into the segmentation model
- Compare the segmentation performance on non-centered portrait images before and after applying your preprocessing pipeline

- **Vision Foundation Model SAM 2. (5 points)**

SAM 2 (<https://github.com/facebookresearch/sam2>) is a vision foundation model that can segment anything without fine-tuning. Apply SAM 2 to portrait segmentation and visualize at least 5 examples showing the input images, prompts, and predicted masks. Briefly compare its zero-shot performance with your trained models.

4.3 Reference

1. PyTorch. (n.d.). Learn the Basics. PyTorch Tutorials. <https://pytorch.org/tutorials/beginner/basics/intro.html>
2. PyTorch. (2024). PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>

5 Submission Guidelines

Your submission must include:

`report.pdf`: A comprehensive project report (maximum 6 pages including references) detailing your work. Only the provided LaTeX template (CVPR style) is allowed. The report should include: Detailed procedures with explanatory images (if needed), final pseudo-code, relevant mathematical theory, your thought process, thorough experiments and results analysis, and **a brief introduction to your code usage (Important!)**.

`code/`: A directory containing all your code for this assignment.

`checkpoints/`: Required checkpoints. For large files, provide a properly shared OneDrive link. Inaccessible links count as non-submission.

Grading: 60% report, 40% code. **Plagiarism is strictly prohibited and will be checked. Ensure your code runs without errors.**

Submission Process: Rename your folder as <your student ID>-<your name>-Asgn1, compress it to <your student ID>-<your name>-Asgn1.zip, and upload to the blackboard system.

Please read these guidelines carefully. Late submissions will incur penalties: 1 day late: -20 marks; 2 days: -40 marks; 3 days: -100 marks.